Problem (3)

Student 1: Saachi Tamboli

Student 2: Brett Luthern

Note: This problem takes especially long to run, with both our computers taking roughly 10 minutes to execute the entire problem.

```
% Parameters

zetaP = 350; %km - perigee altitude
rp = 6371 + zetaP; % perigee radius
e = 0.1;
Omega = 195 * pi/180; %rad
i = 54 * pi/180; %rad
w = 235 * pi/180; %rad
ta = 20 * pi/180; %rad
muE = 3.986e5;
rE = 6371;

N = 30;
sigma_v = 0.1; % km/s  (= 100 m/s)
mu_bc = log(20);
sigma_bc = 0.5;
```

Part A:

In this part, we are asked to code a function explosion that generates the (3XN) array v and the (1XN) array bc with the initial velocities and the ballistic coefficients of N pieces of degree, and inserting $\mu_v, \mu_{\text{bc}}, \sigma_v$ & σ_{bc} . Our first step was to initialize all relevant parameters, shown above. We then calculate a using the following equation:

$r_p = a(1-e)$ which can be rearranged to be $a = \dfrac{r_p}{1-e}$

Once a is acquired, we convert the 6 classical orbital elements into cartesian coordinates, using a previously obtained and explained function. This gives r_0 and v_0.

The first step of our function is to define previously initialized parameters that were not requested to be inputs of the function (N). We then are able to calculate v using the equation below, giving each fragment a random velocity:

$v_i = v_{\text{sat}} + \delta v_i$

We then use the lognormal function in matlab to obtain ballistic coefficient, as described in the problem description. Using the requested rng seed of one, we run the function using the initialized parameters. the initial position of all 30 debris are then mapped with the same position in r0_debris.

```
a = rp/(1-e);
[r0_sat, v0_sat] = coe2rv(a, e, Omega, i, w, ta, muE);
```

```
r0_sat = r0_sat(:);  v0_sat = v0_sat(:);
% a) Code function [v, bc] = explosion(mu_v, sigma_v, mu_bc, sigma_bc)

function [v, bc] = explosion(mu_v, sigma_v, mu_bc, sigma_bc)
    N  = 30;
    v  = mu_v(:) + sigma_v * randn(3, N);
    bc = lognrnd(mu_bc, sigma_bc, [1, N]);
end

rng(1);
[v0_debris, bc] = explosion(v0_sat, 0.1, mu_bc, sigma_bc);

N = 30;
V = zeros(3, N);

r0_debris = repmat(r0_sat(:), 1, N); % map all 30 with the same position
```

Part B:

Part b asks to implement a function drag_acceleration that returns the perturbation acceleration vector due to drag on an object. After initializing parameters, we establish an if statement that cuts off the altitude before 100 km. Below that altitude, the atmosphere is too dense and our model breaks, so we set values below the threshold to be NaN

We know that the drag equation is as follows:

$F_d = -\frac{1}{2}\rho C_d A|v|v$, and from the slides this can be rewritten as:

$$a_p = -\frac{1}{2}\frac{\rho}{\mathrm{BC}}|v|v$$

We also know ρ to be $\rho = \rho_0 e^{-\frac{(\zeta-\zeta_0)}{H}}$.

Now that all equations ave been identified, the function calculates ρ and $a_{p,\mathrm{drag}}$ using a unit conversion.

```
function ap = drag_acceleration(zeta, v, bc)
% Isothermal atmosphere drag. v in km/s, bc in kg/m², returns km/s².
    H     = 50;        % scale height [km]
    rho0  = 1e-8;      % density at zeta0 [kg/m³]
    zeta0 = 100;       % reference altitude [km]
    if zeta < zeta0
        ap = nan(3,1);
        return
    end
    rho = rho0 * exp(-(zeta - zeta0) / H);    % kg/m^3

    % Unit conversion to km/m^3 multiply by 1000 to get km/s².
    ap = -0.5 * rho * norm(v) * v(:) / bc * 1000;
```

```
end
```

Part C:

Part C asks us to generate a function that returns the perturbation acceleration vector due to the J_2 coefficient of the non-sphericity of the Earth. We initialize Earth's oblateness coefficient, the gravitational parameter and and Earth radius, along with the input of the function. It's known that the gravity potential is a more complex equation due to Earth it being a perfect sphere, becoming the equation as follows:

$$U = -\frac{\mu}{r}\left(1 - J_2\left(\frac{R_E}{t}\right)^2 P_2 \sin(\phi)\right)$$

We then extract the position components into their on variables from the inputted r variable, finds the magnitude, and calculated how far the debris is from the equatorial plane. We the calculate for a_{J2} using the following formula:

$$a_{J2} = -\frac{3}{2} J_2 \mu \frac{R_E}{r^5}\begin{bmatrix} x\left(1 - 5\left(\frac{z}{r}\right)^2\right) \\ y\left(1 - 5\left(\frac{z}{r}\right)^2\right) \\ z\left(1 - 5\left(\frac{z}{r}\right)^2\right) \end{bmatrix}$$

ending the part by calculating the for a_{J2} at the initial position.

```
function ap = J2_acceleration(r)
    J2  = 0.00108263;
    muE = 3.986e5;
    rE  = 6371;
    x = r(1);  y = r(2);  z = r(3);
    rmag = norm(r);
    rz2  = (z/rmag)^2;
    factor = -1.5 * J2 * muE * rE^2 / rmag^5;
    ap = factor * [ x*(1 - 5*rz2);
                    y*(1 - 5*rz2);
                    z*(3 - 5*rz2) ];
end
```

Part D:

Part D asks us to establish a Cowell propagator that integrates the trajectories of multiple objects, with arrays r0 and v0 giving their initial positions and velocities. Cowell's method directly integrates the following:

$$\frac{d}{\mathrm{dt}}\begin{bmatrix} r \\ v \end{bmatrix} = \begin{bmatrix} v \\ a(r, v) \end{bmatrix}$$

From the slides, we gather that $a_B = -\frac{\mu}{r^3}|r| + \text{other forces}$, so our goal is to propagate each debris using $(\ddot{r}) = -\frac{\mu}{r^3}|r| + a_{J2} + a_{\text{drag}}$ using the functions obtained in previous parts.

The first step is to initialize values within the function that were not requested as inputs. We then begin a for loop that allows us to complete calculations for each debris particle. We then establish an initial state for each r0 and v0 value within a new y0 variable, and use the Runge-Kutta method in order for the simulation to include nonlinear and velocity-dependent forces. This function is then able to produce the requested [3xNxM] matrices.

We then begin a new function, eom, that begins by calculating the central gravitational force $a_{\text{grav}} = -\frac{\mu}{r^3}|r|$. Through multiple if statements, this function accordingly adds the perturbations of drag and J2 if it meets the criteria of the if statement, and outputs the state derivative $\frac{d}{\text{dt}}\begin{bmatrix} r \\ v \end{bmatrix} = \begin{bmatrix} v \\ a \end{bmatrix}$. In a final function, we define a function that stops the integration when the altitude of the debris drops below 100 km.

```
function [r, v] = cowell(r0, v0, bc, t, flag_drag, flag_J2)
    muE = 3.986e5;
    rE  = 6371;
    N   = size(r0, 2);
    M   = numel(t);
    r   = nan(3, N, M);
    v   = nan(3, N, M);

    opts = odeset('RelTol', 1e-9, 'AbsTol', 1e-10, ...
                  'Events', @(tt,y) reentry_event(tt,y,rE));

    for k = 1:N
        y0 = [r0(:,k); v0(:,k)];
        try
            [Tsol, Y, te, ~, ~] = ode45( ...
                @(tt,y) eom(tt,y,bc(k),muE,rE,flag_drag,flag_J2), ...
                t, y0, opts);
        catch
            continue
        end

        % If the event triggered, ode45 returns Tsol with rows up to and
        % including the event time.
        nT = numel(t);
        for idx = 1:size(Y,1)
            % Find which requested time this row matches (if any)
            [dt_min, j] = min(abs(t - Tsol(idx)));
            if dt_min < 1e-6 * max(abs(t(end)), 1)
                r(:, k, j) = Y(idx, 1:3).';
                v(:, k, j) = Y(idx, 4:6).';
            end
        end
```

```
    end
end

function dy = eom(~, y, bc_k, muE, rE, flag_drag, flag_J2)
    rvec = y(1:3);  vvec = y(4:6);
    rmag = norm(rvec);
    accel = -muE / rmag^3 * rvec;
    if flag_J2
        accel = accel + J2_acceleration(rvec);
    end
    if flag_drag
        zeta = rmag - rE;
        if zeta >= 100
            ad = drag_acceleration(zeta, vvec, bc_k);
            if ~any(isnan(ad))
                accel = accel + ad;
            end
        end
        % If below 100 km, don't add drag
    end
    dy = [vvec; accel];
end

function [value, isterminal, direction] = reentry_event(~, y, rE)
% Stops integration when altitude drops to 100 km
    rmag = norm(y(1:3));
    value = (rmag - rE) - 100;     % zero when altitude = 100 km
    isterminal = 1;                % stop on detection
    direction  = -1;               % only when descending
end
```

Part E:

Tying everything together, part E asks to simulate the trajectories of the cloud of debris for 6 months in multiple cases: without perturbations, with the drag perturbation and with both perturbations, for each case showing various plots.

We first initialized our variables, generating a range of days that were turned into seconds per the problem's instructions. We then establish three cases, in line with the cases requested. Case one is with no perturbations (i.e. false false), case two is with the drag force, and case three is with both perturbations included.

We then use the same of 1 and run the explosion function, resetting the r0 and v0 values after the function is run. We then propagate using the cowell function generated in Part D, and extract the orbital elements at each epoch into their own respective variables. For each debris we convert the obtained r v values back to classical orbital elements using a previously explained rv2coe function, and begin plotting and printing plots.

In the first case, the Gabbard plot presents a static shape, with both the a vs. e and Ω vs. a plots showing constant motion. With no perturbations we see the debris cloud initially spread due to velocity dispersion, but

with only gravitational forces present the orbital elements remain constant and justifies the results seen in the first 3 charts.

In the second case, we see a Gabbard plot with points that shift downward. We also see that a values decrease over time and that the overall spread of the debris field widens as time goes on. This is because the perturbations due to drag reduce the orbital energy over time, causing a decrease in the semi-major axes and orbital periods in Case 2 plots. This also causes differences in ballistic coefficients, leading to an increased dispersion in the debris cloud.

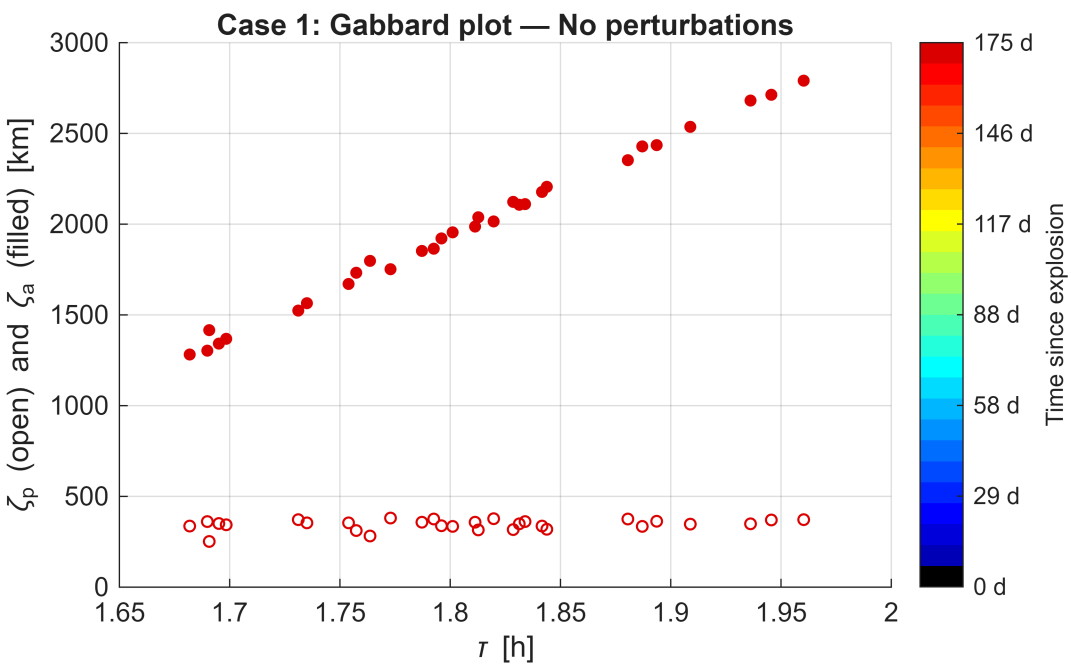
For the third and final case, we continue to see the downward shifting, but with even wider spread of the cloud in the Gabbard plot. We also now see that Ω changes over time. The addition of J2 introduces changes in the orbital elements, specifically in the right ascension of the ascending node. Drag is continuing to reduce the semi major axes, but J2 causes even more dispersion in the debris cloud, resulting in a more complex evolution as time progresses compared to the drag only case, and considerably different compared to the case with zero perterbations considered.

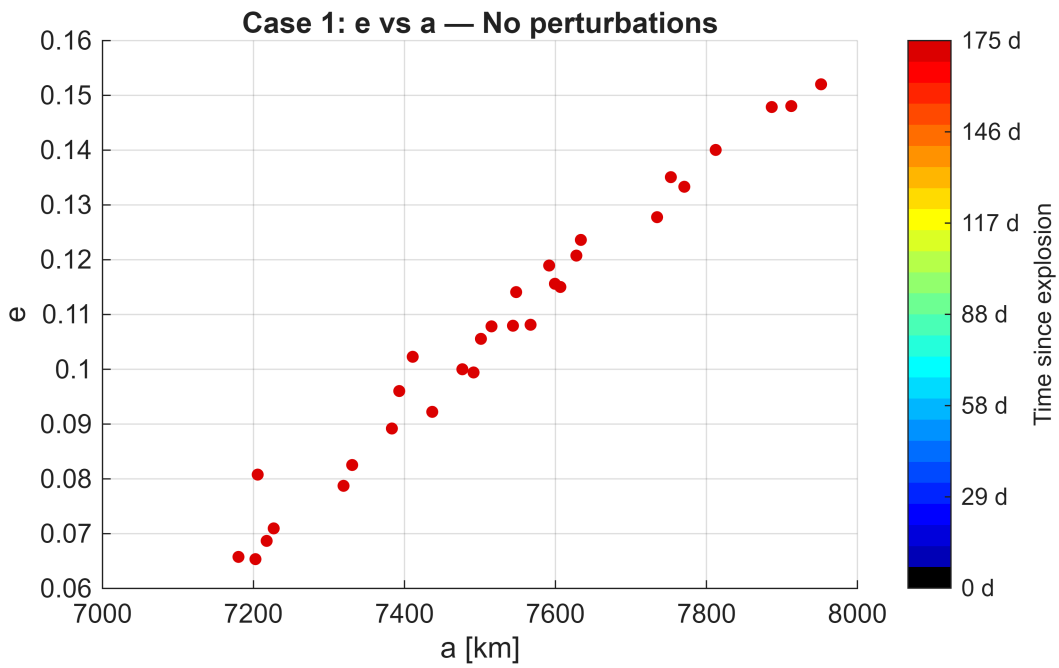
```
dt_days = 7;
t_days  = 0:dt_days:180;     % 0, 7, 14, …, 180  (27 epochs)
t_sec   = t_days * 86400;
M = numel(t_sec);

case_titles = {'No perturbations', 'Drag only', 'Drag + J2'};
case_flags  = {[false false], [true false], [true true]};

for c = 1:3
    flag_drag = case_flags{c}(1);
    flag_J2   = case_flags{c}(2);

    fprintf('\n=== Case %d: %s ===\n', c, case_titles{c});

    rng(1);
    [v_dispersion, bc] = explosion(v0_sat, sigma_v, mu_bc, sigma_bc);
    v0_debris = v_dispersion;
    r0_debris = repmat(r0_sat, 1, N);

    fprintf('Propagating %d pieces over %d epochs...\n', N, M);
    tic;
    [r_traj, v_traj] = cowell(r0_debris, v0_debris, bc, t_sec, flag_drag, flag_J2);
    fprintf('  Cowell: %.2f s\n', toc);

    % Extract orbital elements at each epoch
    a_hist     = nan(N, M);
    e_hist     = nan(N, M);
    Omega_hist = nan(N, M);
    zp_hist    = nan(N, M);
    za_hist    = nan(N, M);
    tau_hist   = nan(N, M);

    for k = 1:N
```

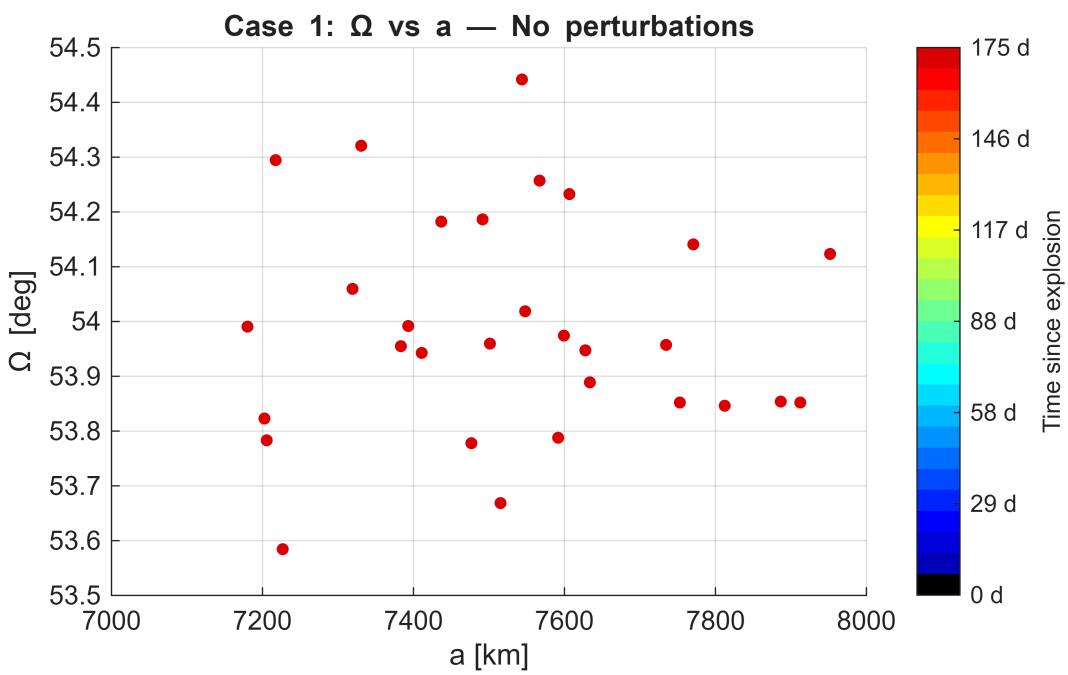
```
        for m = 1:M
            rk = r_traj(:,k,m);
            vk = v_traj(:,k,m);
            if any(isnan(rk)) || any(isnan(vk));  continue;  end
            [aa, ee, OO, ~, ~, ~] = rv2coe(rk, vk);
            if ~isfinite(aa) || aa <= 0 || ee >= 1;  continue;  end
            a_hist(k,m)     = aa;
            e_hist(k,m)     = ee;
            Omega_hist(k,m) = OO;
            zp_hist(k,m)    = aa*(1-ee) - rE;
            za_hist(k,m)    = aa*(1+ee) - rE;
            tau_hist(k,m)   = 2*pi*sqrt(aa^3/muE) / 3600;
        end
    end

    % ── Plotting (all 3 figures per case) ──────────────────────────
    % Color map: t=0 black, then jet from blue (early) to red (late)
    cmap = [0 0 0; jet(M-1)];

    % Gabbard plot — perigee (○) and apogee (●) altitude vs period
    figure('Name', sprintf('Case %d Gabbard', c), 'Color', 'w');
    hold on; grid on;

    for m = 1:M
        col = cmap(m,:);
        valid = isfinite(tau_hist(:,m));
        if ~any(valid); continue; end          % skip empty epochs
        h1 = plot(tau_hist(valid,m), zp_hist(valid,m), 'o', ...
        'MarkerEdgeColor', col, 'MarkerSize', 4, 'LineWidth', 0.8);
        plot(tau_hist(valid,m), za_hist(valid,m), 'o', ...
        'MarkerFaceColor', col, 'MarkerEdgeColor', col, 'MarkerSize', 4);
        if ~isempty(h1)
            leg_handles(m) = h1(1);            % h1 may be vector — take first
        end
    end
    xlabel('\tau [h]'); ylabel('\zeta_p (open) and \zeta_a (filled) [km]');
    title(sprintf('Case %d: Gabbard plot — %s', c, case_titles{c}));
    add_time_colorbar(t_days, cmap);

    % e vs a
    figure('Name', sprintf('Case %d e vs a', c), 'Color', 'w');
    hold on; grid on;
    for m = 1:M
        col = cmap(m,:);
        valid = isfinite(a_hist(:,m));
        plot(a_hist(valid,m), e_hist(valid,m), 'o', ...
            'MarkerFaceColor', col, 'MarkerEdgeColor', col, 'MarkerSize', 4);
    end
    xlabel('a [km]'); ylabel('e');
    title(sprintf('Case %d: e vs a — %s', c, case_titles{c}));
```

```
    add_time_colorbar(t_days, cmap);

    % Omega vs a
    figure('Name', sprintf('Case %d Omega vs a', c), 'Color', 'w');
    hold on; grid on;
    for m = 1:M
        col = cmap(m,:);
        valid = isfinite(a_hist(:,m));
        plot(a_hist(valid,m), rad2deg(Omega_hist(valid,m)), 'o', ...
            'MarkerFaceColor', col, 'MarkerEdgeColor', col, 'MarkerSize', 4);
    end
    xlabel('a [km]'); ylabel('\Omega [deg]');
    title(sprintf('Case %d: \\Omega vs a — %s', c, case_titles{c}));
    add_time_colorbar(t_days, cmap);
end
```

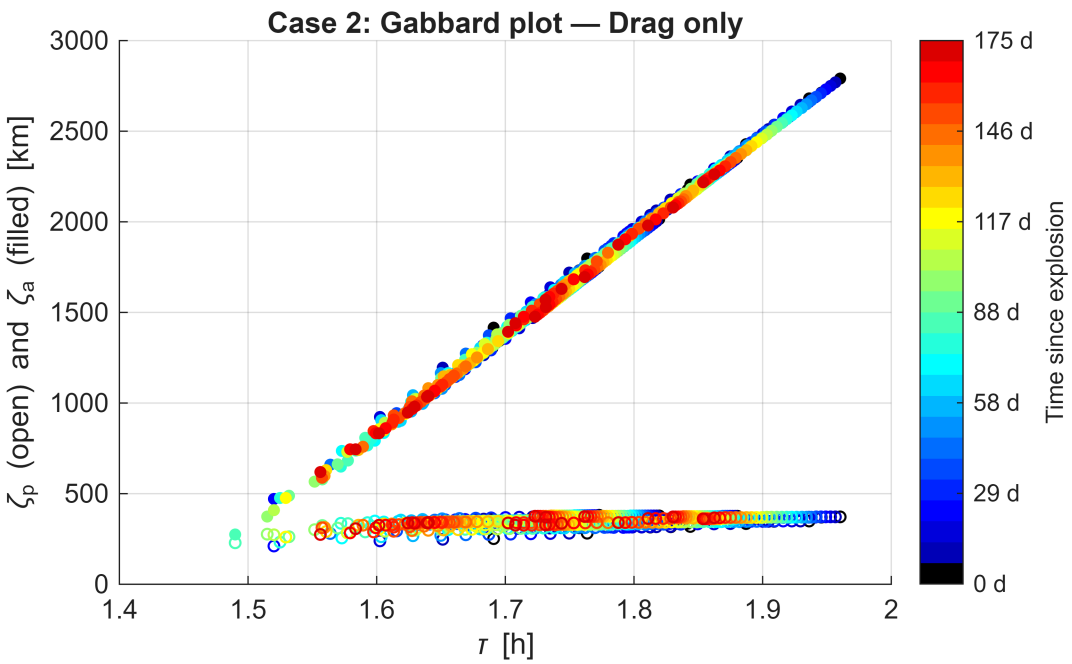
```
=== Case 1: No perturbations ===
Propagating 30 pieces over 26 epochs...
  Cowell: 422.61 s
```

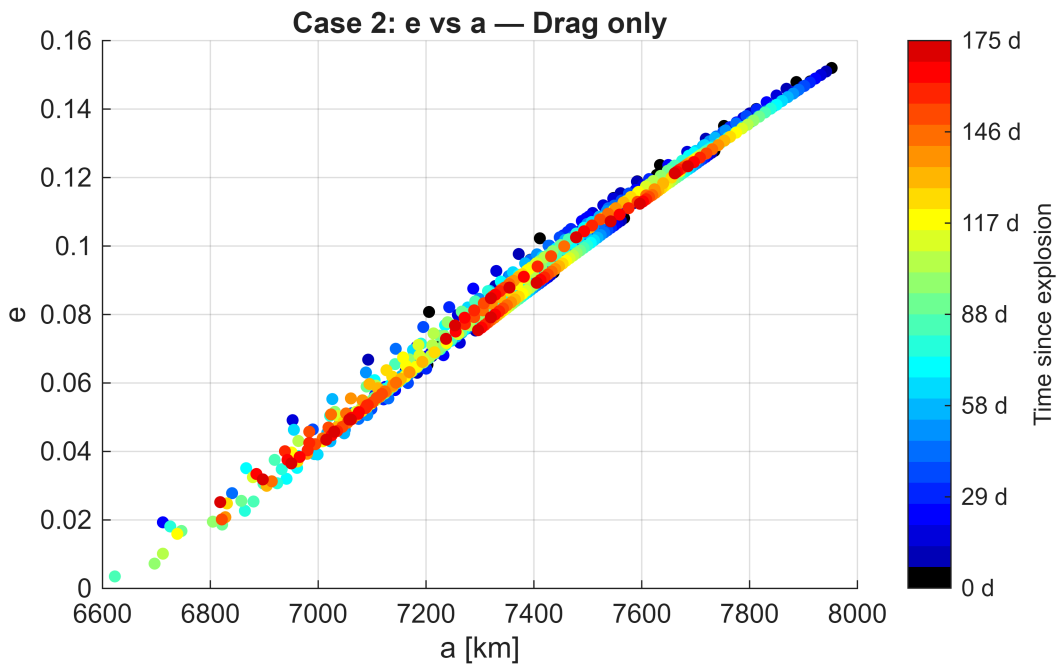


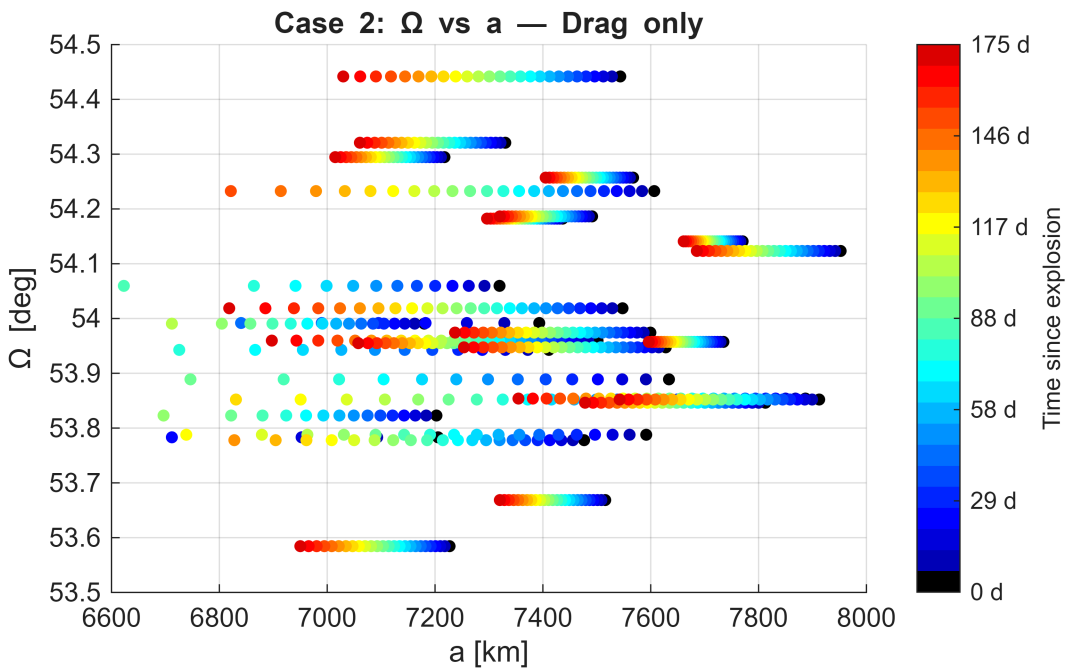




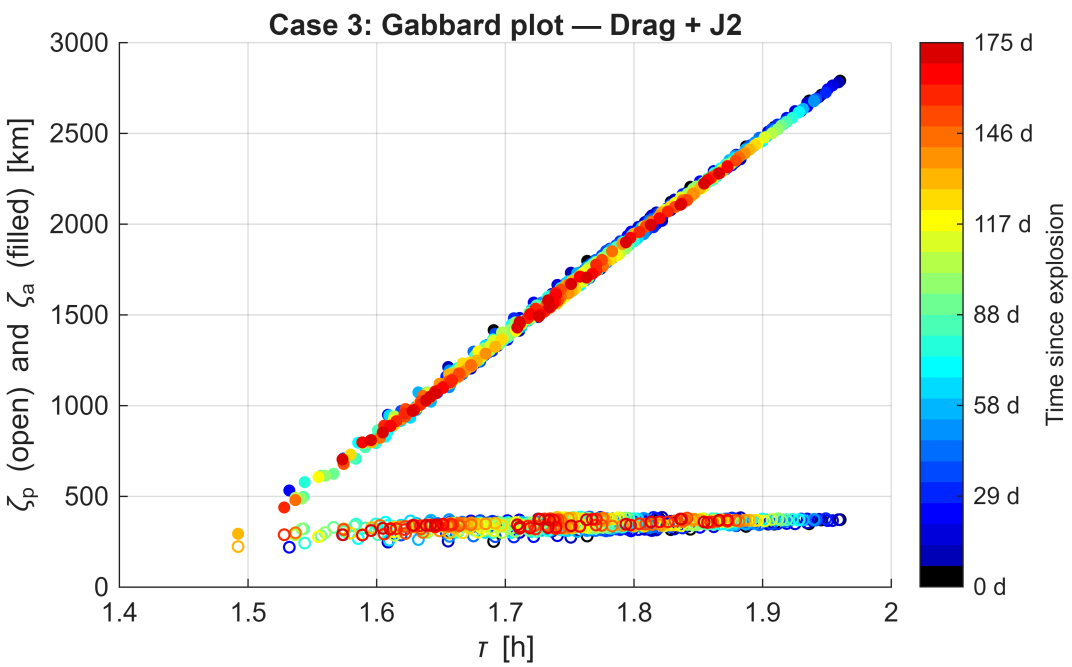

```
=== Case 2: Drag only ===
Propagating 30 pieces over 26 epochs...
  Cowell: 337.76 s
```

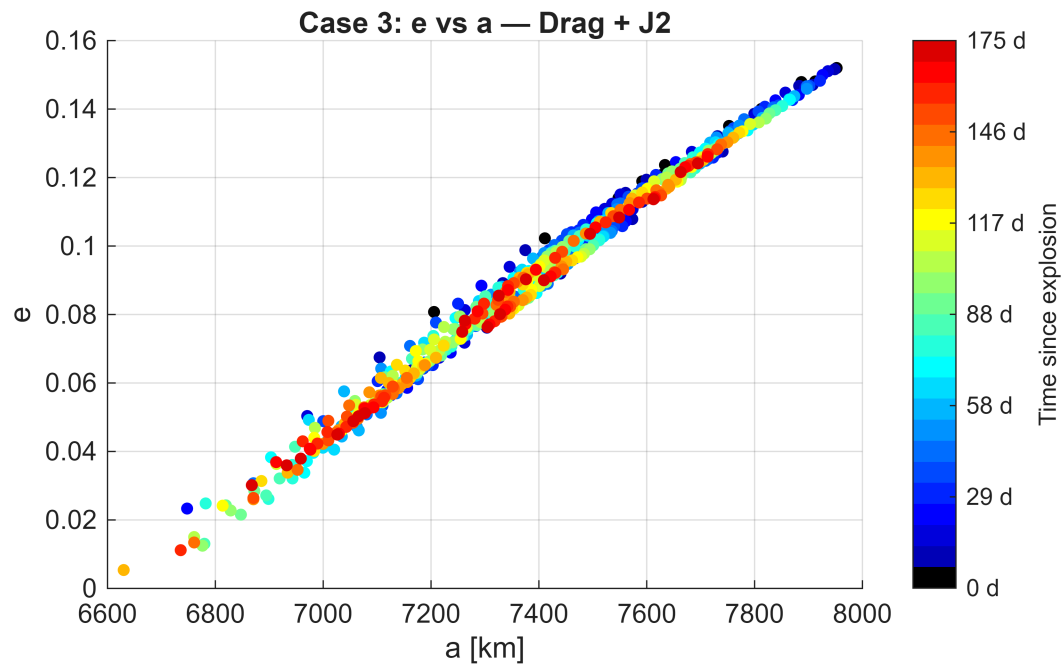

Case 2: Gabbard plot — Drag only
ζ_p (open) and ζ_a (filled) [km]
r [h]
Time since explosion
175 d
146 d
117 d
88 d
58 d
29 d
0 d

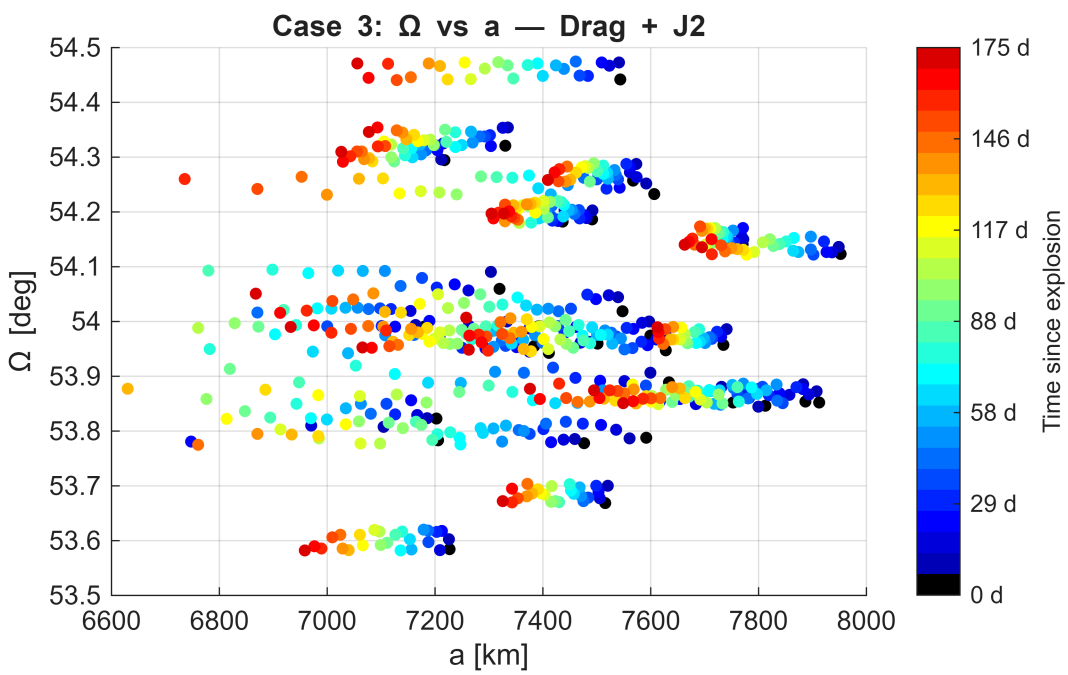

Case 2: e vs a — Drag only
e
a [km]
Time since explosion
175 d
146 d
117 d
88 d
58 d
29 d
0 d




```
=== Case 3: Drag + J2 ===
Propagating 30 pieces over 26 epochs...
  Cowell: 324.74 s
```








```
% color bar function for loop
function add_time_colorbar(t_days, cmap)
% Add a colorbar showing the time of each snapshot.
    colormap(cmap);
    cb = colorbar;
    M  = numel(t_days);
    nticks = min(7, M);
    cb.Ticks = linspace(0, 1, nticks);
    cb.TickLabels = arrayfun(@(x) sprintf('%d d', round(x)), ...
        linspace(t_days(1), t_days(end), nticks), 'UniformOutput', false);
    cb.Label.String = 'Time since explosion';
end
```

Copilot use:

With the main deliverable of this problem being the plots, the use of Ai was used considerably more to help generate plots and print values throughout the parts, specifically in Part E. We also experimented with other Ai's to help debug the process used during this section that copilot was not able to assist with, mostly in Parts D and E. We used ai to create a function that created progress checks to help us understand where the code was faltering during the longer run times. In conclusion, while copilot assisted in the plotting and and output generation, we used other ai to help with debugging. The more detailed failure analysis does help in some cases, but other ai providing deeper and more complex solutions to the code was much more helpful than was copilot was producing.